

Custos / Clew 종합 사업 보고서

프로젝트 전체 대화 기록 분석 · 사업 평가 · 향후 방향성 · 투자 유치 가능성

멀티에이전트 AI 토큰 낭비 탐지 옹저버빌리티

작성일: 2026년 6월 12일

대상 자료: 프로젝트 내 대화 5건 및 관련 외부 문헌

목차

목차	2
1. 요약 (Executive Summary).....	4
2. 한눈에 보기 — 핵심 숫자	5
3. 회사와 제품 — Custos와 Clew	6
3.1 무엇을 만드는가	6
3.2 이름의 유래	6
3.3 진짜 차별점 — 자기기만 방지 규율	6
4. 대화별 상세 기록 — 무슨 일이 있었나.....	8
4.1 대화 ① “-” (6월 2일) — EmergenceTrace의 탄생과 비판	8
4.2 대화 ② “Claude CLI 검증 실험 방법” (~6월 6일) — v1의 정직한 실패와 피벗	9
4.3 대화 ③ “Custos set.1” — 빌드 1부: 데이터 기반과 첫 자기기만 적발	12
4.4 대화 ④ “Custos set.2” — 빌드 2부: 합격까지의 마지막 고비들	13
4.5 대화 ⑤ “Custos 프로젝트 종합 현황 분석 및 문서화” (6월 12일).....	15
5. 기술 심층 해설.....	17
5.1 캐스케이드 아키텍처 — 왜 이 구조인가.....	17
5.2 네 가지 낭비 패턴.....	17
5.3 자기기만 방지 장치 — 이 프로젝트의 영혼	18
5.4 최종 결과 상세	18
6. 관련 논문 및 문서	21
6.1 핵심 외부 논문	21
6.2 내부 핵심 문서 (프로젝트 폴더)	21
7. 시장과 경쟁	23
7.1 시장 규모와 성장	23
7.2 경쟁 지형.....	23
8. 이 사업에 대한 솔직한 의견.....	25
8.1 강점.....	25
8.2 약점과 리스크.....	25
8.3 종합 판단	26
9. 다음 방향 — 앞으로의 로드맵.....	27
9.1 즉시 (1~2주): 실제 트레이스 첫 접촉.....	27
9.2 단기 (2~6주): 첫 사용자와 실측 숫자.....	27
9.3 중기 (6주~3개월): 제품화와 v2.....	27

9.4 전략 원칙 (변하지 않는 것).....	27
10. 투자 유치 가능성 분석 — YC를 중심으로.....	29
10.1 시장의 투자 분위기	29
10.2 YC 관점에서 본 Clew	29
10.3 YC 외 옵션.....	30
10.4 투자 준비 체크리스트.....	30
11. 결론.....	32

1. 요약 (Executive Summary)

Custos(회사)는 **Clew(제품)**라는 멀티에이전트 AI 옴저버빌리티 도구를 만드는 1인 스타트업이다. Clew는 여러 AI 에이전트가 협업할 때 같은 일을 반복하거나 의미 없이 주고받으며 토큰(=비용)을 태우는 "낭비 구간"을 자동으로 찾아내 비용을 줄여주는 것을 목표로 한다.

2026년 6월 2일부터 6월 12일까지 약 열흘간, 이 프로젝트는 다음의 전체 사이클을 완주했다:

아이디어 비판 → 사전등록 검증 실험 → 정직한 실패(v1 폐기) → 피벗 → 시장조사 → 제품 빌드(Stage 1-2) → 사전등록 기준의 단발 평가 통과(GO).

최종 결과: 3가지 낭비 패턴을 탐지하는 구조→의미 캐스케이드 탐지기가, 파라미터 튜닝에 전혀 쓰지 않은 평가 세트(seed=42)에서 **F1 0.8571, 오탐(거짓 양성) 0건, 범위 내 탐지율 100%**로 사전등록된 합격 기준(F1 0.80 이상)을 통과했다. 단, 이는 전부 합성(인공 생성) 데이터 위의 결과이며, 실제 프로덕션 트레이스 검증·실측 토큰 절감액·사용자 확보는 모두 다음 단계의 과제로 남아 있다.

쉽게 말하면

열흘 동안 "아이디어를 의심하고 → 실험으로 죽여보고 → 살아남은 방향으로 다시 만들어서 → 시험을 통과시키는" 한 바퀴를 다 돌았습니다. 시험은 통과했지만 아직 '모의고사'(인공 데이터)입니다. 이제 '실전'(진짜 회사들의 AI 로그)을 치러야 하는 단계입니다.

Custos/Clew 프로젝트 여정 (2026년 6월)



2. 한눈에 보기 — 핵심 숫자

항목	값	의미
회사 / 제품	Custos / Clew	수호자(라틴어) / 아리아드네의 실타래
v1(EmergenceTrace) 결과	AUC 0.455	동전 던지기 수준 → 정직하게 폐기
v1 핵심 교란	길이 상관 $r=0.856$	신호가 사실은 '트레이스 길이'를 측정
Stage 1 동결	80 트레이스, 92 테스트	commit 0fa25e0, tag stage1-freeze
동결 파라미터	$\phi=0.514345$, $N=2$	dev(seed=7)에서만 결정
dev 캘리브레이션	gap +0.2208, d=4.38	분포 분리 기준 C1-C4 전부 통과
최종 단발 평가	F1 0.8571, FPR 0.0	seed=42, 단 1회만 접촉 — GO
범위 내 탐지율	30/30 (100%)	3패턴: repeat-pingpong-requery
범위 밖 패턴	regen_handoff 0/10	원리적 사유로 v1에서 의도적 제외
테스트 수	146개 통과	1건은 의도된 xfail
시장 규모	\$2.69B → \$9.26B	2026→2030, 연평균 약 36% 성장
기업 AI 중 멀티에이전트	72%	2024년 23%에서 급증
실사용자 / 실측 절감액	0명 / 0건	다음 단계의 최우선 과제

쉽게 말하면

위에서 가장 중요한 두 줄은 맨 아래 두 줄입니다. 기술 시험은 만점에 가깝게 통과했지만, '진짜 고객의 데이터에서 돈을 얼마나 아껴줬는가'라는 사업의 핵심 숫자는 아직 0입니다. 투자자도 이 숫자부터 물어봅니다.

3. 회사와 제품 — Custos 와 Clew

3.1 무엇을 만드는가

여러 개의 AI 에이전트(LLM)가 팀을 이뤄 일하는 '멀티에이전트 시스템'이 기업 AI의 주류가 되고 있다. 그런데 에이전트들은 종종 (1) 같은 노드가 같은 일을 반복하고, (2) 두 에이전트가 결론 없이 핑퐁을 주고받고, (3) 이미 조회한 정보를 또 조회하고, (4) 앞 에이전트가 만든 결과를 뒤 에이전트가 다시 만들어낸다. 이 모든 것이 토큰을 태우는 순수한 낭비이며, 토큰 비용 폭증은 현재 기업들이 꼽는 1순위 운영 고통이다.

Clew는 멀티에이전트 실행 기록(트레이스)을 받아 이런 낭비 구간을 자동으로 짚어주고, 어디서 얼마나 토큰이 새는지 보고하는 도구다. 분류상 'LLM 옹저버빌리티' 시장에 속하되, 범용 모니터링이 아니라 "낭비 탐지로 비용 절감"이라는 한 가지 쐐기(wedge)에 집중한다.

3.2 이름의 유래

- **Custos** — 라틴어로 '수호자·파수꾼'. 회사의 지속 명제(시스템을 지켜본다)를 담는다.
- **Clew** — 그리스 신화에서 아리아드네가 테세우스에게 준 '실타래'. 미로에서 길을 찾게 한 그 실이며, 영어 단어 clue(단서)의 어원이다. 에이전트 호출의 미로에서 길을 추적한다는 제품 정체성과 정확히 맞는다.
- 이 짝은 Anthropic(미션을 담은 어근) + Palantir(기능을 담은 신화)식 네이밍 구조를 따른 것이다. 후보였던 Pharos 는 기존 AI 관측 도구(pharos.ai)와 충돌해 제외했고, Cairn·Vigil 등 대안 중 Clew 가 최종 선택됐다. 상표·도메인 확정 검증은 아직 직접 수행해야 할 항목으로 남아 있다.

3.3 진짜 차별점 — 자기기만 방지 규율

이 회사의 가장 강한 차별점은 기능 목록이 아니라 **anti-self-deception(자기기만 방지)** 엔지니어링 규율이다. 결과를 보기 전에 합격/불합격 기준을 미리 등록하고, 평가 데이터는 단 한 번만 건드리고, 통과하더라도 그 통과가 '진짜'인지 스스로 의심하는 절차를 코드와 문서로 강제했다. 이 규율 덕분에 (1) v1을 매몰비용 없이 빠르게 죽일 수 있었고, (2) Stage 2에서 '통과하는 수정'

이 사실은 어려운 케이스를 숨긴 자기기만임을 잡아냈으며, (3) 출시 직전 점검에서 한 패턴이 전혀 탐지되지 않는 구멍을 발견했다.

쉽게 말하면

회사 이름은 '지키는 자', 제품 이름은 '미로를 빠져나오게 한 실'입니다. 그리고 이 회사가 정말로 남다른 점은, 자기 자신이 만든 결과조차 의심하고 검증하는 습관을 시스템으로 만들어 놨다는 것입니다. AI가 코드를 짜주는 시대에는 '그렇듯하게 통과한 것처럼 보이는 결과'가 가장 위험한데, 그 위험을 정면으로 관리합니다.

4. 대화별 상세 기록 — 무슨 일이 있었나

프로젝트에는 Custos SNS를 제외하고 총 5건의 대화가 있다. 시간순으로 보면 하나의 연속된 여정이다: 아이디어 비판 → 검증 실험과 정직한 실패 → 피벗과 전략 → 제품 빌드 1부 → 제품 빌드 2부와 최종 합격 → 문서화.

4.1 대화 ① “-” (6 월 2 일) — EmergenceTrace 의 탄생과 비판

시작은 “이 사업에 대해서 너의 생각은 뭐야”라는 질문이었다. 당시 사업의 이름은 **EmergenceTrace**였고, 핵심 베팅은 “기존 옹저버빌리티 툴은 전부 개별 에이전트만 보고, 에이전트들 사이의 창발(emergent behavior)은 아무도 안 본다”는 것이었다. 정체성은 사후 대시보드가 아니라 실패 직전을 감지해 멈추는 ‘실시간 서킷브레이커’였다.

웹 검색으로 전제를 검증한 결과, 핵심 베팅이 2026년 시장 현실과 정면으로 충돌한다는 점이 드러났다:

- 에이전트 간 관측 — Datadog 이 이미 수행(DASH 2025 에서 실행 흐름 차트 발표, LangGraph·CrewAI·OpenAI Agent SDK 자동 계측).
- Goal Drift / 무한루프 감지 — Galileo 가 ML 기반으로 수행, Confident AI 도 역할 드리프트·에스컬레이션 실패 탐지.
- ‘창발 행동 관측’이라는 학술 컨셉 자체가 이미 경쟁사의 마케팅 언어가 됨.

경쟁사 7개(LangSmith, AgentOps, Arize Phoenix, Datadog, Galileo, Confident AI, EmergenceTrace)의 기능을 칸별로 재검증해 정직한 비교표를 다시 만들었고, 결론은 EmergenceTrace가 혼자 우위인 칸은 ‘단일 정량 Emergence Score’ 단 한 줄뿐이라는 것이었다. 동시에 시장이 세 진영(APM 계열 Datadog·New Relic / AI 네이티브 추적 Langfuse·LangSmith / 게이트웨이 Helicone·Portkey)으로 나뉘는데 EmergenceTrace의 포지션이 애매하다는 점도 지적됐다.

그럼에도 시장 자체는 진짜였다: 조직의 89%가 에이전트 옹저버빌리티를 도입했고, 품질 문제가 32%로 최대 프로덕션 장벽이며, EU AI Act 규제 압력도 실재했다. 살릴 방향으로 (1)

Emergence Score가 진짜 차별점이면 거기에 전부 걸기, (2) '프레임워크 무관'은 차별점이 아니라 입장권이라는 인식, (3) 수평에서 수직(고위험 감사 시장)으로 좁히기가 제시됐다.

이 대화의 산출물은 두 가지다: 1쪽짜리 실험 설계서와, 17개 참고문헌을 갖춘 정식

EmergenceTrace 연구보고서(문제 정의, 관련 연구, 경쟁 지형, Emergence Score 방법론, 9개 하위 절의 기술 아키텍처, 검증 계획, 시장 맥락, 리스크 분석). Emergence Score는 '시스템 전체 행동에서 개별 에이전트 행동의 합을 뺀 잔차(residual)'로 정식 정의됐고, UC Berkeley의 MAST-Data 벤치마크로 검증 실험을 하기로 설계됐다.

쉽게 말하면

첫 대화에서 '아무도 안 하는 일'이라는 전제가 사실이 아님이 드러났습니다. 하지만 사업을 접는 대신, 유일하게 남은 차별점(Emergence Score)이 진짜인지 실험으로 확인하기로 했습니다. 이 '확인부터 한다'는 결정이 이후 모든 것의 출발점입니다.

4.2 대화 ② “Claude CLI 검증 실험 방법” (~6 월 6 일) — v1의 정직한 실패와 피벗

4.2.1 검증 실험의 설계

검증 가설: “두 신호(Loop Pressure, Information Gain Decay)만으로 멀티에이전트 실패를 사고 전에, 개별 에이전트 합산 방식보다 더 일찍·더 적은 오탐으로 예측할 수 있다.” 참이면 사업이 있고, 거짓이면 한 달 안에 알고 피벗한다는 원칙이었다.

- **Loop Pressure(LP)** — 에이전트 간 메시지 그래프의 사이클(왕복·삼각)을 잡되, 정보가 빈곤할수록 가중. '팀이 빙빙 도는 압력'.
- **Information Gain Decay(IGD)** — 임베딩 기반으로 스텝당 새 정보량이 마르는 속도를 측정. '새 정보가 고갈되는 신호'.
- **데이터** — UC Berkeley MAST-Data(Hugging Face mcmri/MAST-Data), 7 개 프레임워크에 걸친 1,242 건의 실제 성공/실패 트레이스.

- **규율** — 결과를 보기 전 $AUC \geq 0.80$ 을 합격 기준으로 사전등록, 모든 변경은 번호 붙은 수정안(AMD)으로 기록, 신호 코드와 라벨을 분리하는 누출 방지 구조.

실험 도중 데이터가 던진 문제도 정직하게 처리됐다: LP를 정의대로 썰 수 있는 프레임워크가 AG2 하나뿐이고 표본도 부족해, 주 검증 대상을 IGD 단독으로 재조정(AMD-005)하고 LP는 '방향성 증거'로 강등했다.

4.2.2 결과 — 깨끗한 음성

- **AUC ≈ 0.455** — 동전 던지기 수준. 사전등록 기준 0.80 에 한참 미달.
- **더 나쁜 발견: 신호가 트레이스 길이와 Spearman $r = 0.856$ 로 상관.** 실패 여부가 아니라 '얼마나 긴가'를 재고 있었다.
- 길이 영향을 제거하고 재집계하니 AUC 0.42, 게다가 방향이 역전 — 성공한 트레이스가 오히려 정보 감쇠가 더 컸다($p \approx 0.013$).

결론은 정확하게 표현됐다: "반증된 것은 아니지만 미검증 — 이 구현이 데이터에서는 음성." 그리고 이 실험 자체는 성공으로 평가됐다. 몇 주 만에, 매몰비용의 늪 없이, 몰래 문자열 길이를 재는 지표 위에 대시보드를 쌓기 전에 정직한 답을 얻었기 때문이다.

4.2.3 피벗 — '실패 예측'에서 '낭비 탐지'로

이후 웹 리서치를 통해 사업의 썩기를 재정의했다: "실패를 예측한다"(검증 실패)가 아니라 "토큰을 태우는 헛도는 중복 루프·핸드오프를 탐지해 잘라낸다" — 검증된 고통(토큰 비용 폭증)에 뿌리내린 비용 스토리다.

이 피벗을 기술적으로 뒷받침한 것이 **IBM Research 논문(arXiv:2511.10650)**이다. 구조 신호가 후보 루프를 찾고 의미 유사도가 중복을 확정하는 '구조→의미 캐스케이드'가 F1 0.72를 달성한 반면, 구조 단독은 0.08, 의미 단독은 0.28에 그쳤다. 이는 v1의 직관(루프·정보 신호)이 틀리지 않았으며 구현이 틀렸음(캐스케이드 대신 합산, 국소 중복 대신 전역 추세)을 보여준다. 단, 이 수치는 외부 연구 결과이지 우리의 검증 결과가 아니며, 모든 공개 커뮤니케이션에서 이 구분을 유지하기로 못 박았다.

4.2.4 시장조사·GTM 준비·네이밍

- 시장: LLM 오피저버빌리티 약 \$2.69B(2026) → \$9.26B(2030), 연평균 약 36% 성장. 기업 AI 의 72%가 멀티에이전트(2024 년 23%에서 급증), 만족도는 33% 미만, 토큰 비용 폭증이 1 순위 운영 고통.
- 경쟁: Langfuse(2026 년 1 월 ClickHouse 가 \$400M 에 인수), LangSmith, Laminar, Arize/Phoenix, Datadog 및 다수의 자금력 있는 스타트업.
- GTM 초안: 각 서브레딧 규칙을 준수한 Reddit 글, Ask HN 글, 정직한 음성 결과 실험기를 담은 DEV.to 전체 기사, Discord 메시지, 한국 커뮤니티(Instruct.KR, 모두의연구소) 글, GitHub 이슈 작성자 대상 콜드 DM 템플릿까지 작성 완료. 6 가지 단골 질문 유형에 대한 가치 우선 댓글 초안과 실제 스레드 3 건에 대한 답글도 준비.
- 빌드 우선 결정: 커뮤니티 검증을 기다리지 않고 빌드로 진행하되, '가장 위험한 가정을 시험하는 가장 작은 진짜 산출물'만 만든다는 조건. 이 시점에 회사 Custos · 제품 Clew 로 네이밍 확정.

쉽게 말하면

v1은 시험에서 떨어졌습니다. 그런데 떨어진 이유를 끝까지 파보니 '신호가 사실 길이를 재고 있었다'는 함정까지 찾아냈고, 덕분에 가짜 성공 위에 회사를 짓는 최악의 시나리오를 피했습니다. 그리고 IBM의 외부 연구에서 '올바른 조립법(캐스케이드)'을 발견해, 같은 재료로 다른 요리(낭비 탐지)를 시작했습니다.

4.3 대화 ③ “Custos set.1” — 빌드 1 부: 데이터 기반과 첫 자기기만 적발

4.3.1 Stage 1 — 탐지 로직보다 검증 장치를 먼저

탐지 코드를 한 줄도 쓰기 전에 데이터 기반과 검증 장치부터 구축했다:

- OpenInference/OTel 호환 정규 스펙 트리 모델을 pydantic 으로 정의(output_text 필수 필드), LangGraph 어댑터 구축.
- 4 가지 낭비 패턴(repeat_node, regen_handoff, pingpong_aba, requery_known) 각각에 대해 양성 40 개·클린 40 개, 총 80 개의 짝지은 라벨 검증 데이터셋 생성.
- 성공/폐기 기준을 CRITERIA_FROZEN.md 에 동결: $GO = F1 \geq 0.80$ 그리고 통제 $FPR \leq 0.10$ / $KILL = F1 < 0.60$ 또는 $FPR > 0.25$ / 회색지대 $N=3$.
- 3 중 누출 가드: AST 정적 분석 + 런타임 프로브 + 탐지 모듈 비어있음 검사 — 탐지 코드가 라벨 파일을 절대 읽을 수 없게 구조로 보장.
- 빌드 중 보정: LLM 재생성 패턴을 바이트 동일 복제가 아닌 현실적 패러프레이즈로 교체, manifest sha256 이 트레이스 본문까지 전이적으로 커버하도록 강화.

Stage 1은 92개 테스트 통과와 함께 commit 0fa25e0, tag stage1-freeze로 git에 동결됐다.

4.3.2 Stage 2 시작 — 캐스케이드와 dev/eval 분리

탐지기는 structural.py → semantic.py → cascade.py 3단 구성. 파라미터 ϕ (코사인 임계값), N (반복 임계값), 임베딩 모델은 dev 세트(seed=7)에서만 결정하고, 평가 세트(seed=42)는 파라미터를 git에 동결한 뒤 단 한 번만 접촉한다는 프로토콜을 세웠다.

첫 번째 중대 보정: 초기 캘리브레이션 구현이 dev 라벨에 대한 F1 그리드서치(=dev 과적합)였음을 적발하고, **분포 분리 기반 선택**으로 교체했다. ϕ 는 중복 쌍과 점진 쌍 분포의 P10/P90 갭 중점으로, 적합도는 Cohen's d로 판정 — 라벨 점수 최적화가 아니라 두 분포가 실제로 갈라지는지를 본다.

4.3.3 1 차 캘리브레이션 실패와 ‘자기기만의 순간’

1차 캘리브레이션은 실패했다(dev_fpr 0.20, gap 음수). 분리 가드 3개가 정확히 작동해 평가 세

트 접근을 차단했다. 원인 진단: requery_known의 클린 쌍이 '같은 스키마·다른 값'(예: 고객A 조회 vs 고객B 조회)이라 정형 스키마의 표면 유사도만으로 코사인 ~0.65까지 올라 임계값을 침투한 것. 10개 트레이스가 같은 쌍을 재사용해 다양성도 0이었다 — 모델 한계가 아닌 데이터 결함.

여기서 이 프로젝트 최고의 장면이 나온다. 첫 번째 '수정'은 클린 풀을 전부 다른 도메인으로 다양화하는 것이었고, 숫자는 PASS로 바뀌었다. 그러나 이것은 **한계를 해결한 것이 아니라 한계를 드러내던 케이스를 제거한 것** — 그대로 출시했다면 '고객 A 조회 → 고객 B 조회' 같은 정당한 작업을 낭비로 오탐하는 탐지기가 나갈 뻔했다. 이 자기기만을 스스로 적발하고 되돌렸다.

진짜 수정은 구조 레이어에 '입력 키 게이트'를 넣는 것이었다: 재조회(tool) 후보는 입력이 원본과 동일할 때만 잡는다. 낭비의 정의적 신호는 '같은 입력'이지 '같은 노드'가 아니기 때문이다. 판별을 (못 하는) 의미 레이어에서 (할 수 있는) 구조 레이어로 라우팅했고, '같은 스키마·다른 값' 케이스를 hard negative로 도로 넣어 게이트가 작동함(후보 0)을 증명했다.

쉽게 말하면

시험 채점기를 먼저 만들고, 채점기를 속일 수 없게 자물쇠(누출 가드)까지 채운 다음에야 본 제품을 만들기 시작했습니다. 그리고 중간에 '점수만 오르게 만드는 가짜 수정'을 한 번 했다가 스스로 잡아냈습니다. 어려운 문제를 시험지에서 빼버리면 점수는 오르지만 실력은 늘지 않는다 — 그걸 알아채고 어려운 문제를 도로 넣은 뒤 진짜로 풀었습니다.

4.4 대화 ④ “Custos set.2” — 빌드 2 부: 합격까지의 마지막 고비들

4.4.1 오탐 취약점 수리와 C1-C4 통과

find_repeat_candidates가 노드 토폴로지(agent_or_node_id)만으로 후보를 거르고 있어, '같은 스키마·다른 값' 조회 쌍(코사인 ~0.65)이 의미 레이어까지 도달해 $\phi=0.514$ 에 걸려 오탐될 수 있는 취약점이 확인됐다. span_kind="tool" 스패에 입력 텍스트 동일성 게이트를 추가하고, [A,B,A]처럼 끼어드는 재조회도 잡도록 '직전'이 아닌 '원본 기준' 비교로 구현했다. 패치 4건 (structural.py 게이트, requery_known.py hard-negative 풀 복원, 테스트, calibrate.py 트레이스

단위 FPR 보고)을 완료하고 146개 테스트를 통과시켰다(1건은 의도된 xfail).

캘리브레이션 전에 C1-C4 기준을 SPEC §8과 CRITERIA_FROZEN.md에 사전등록했고, 실행 결과 전부 통과: C1(어려운 클린 → 구조 후보 0), C2(양성 → 정확한 낭비 스펠 플래그), C3(분리: gap +0.2208, Cohen's d 4.3803, 쌍 단위 dev_fpr 0.00), C4(트레이스 단위 FPR 0.00). 동결 파라미터는 $\phi=0.514345$, $N=2$, 임베딩 모델 paraphrase-multilingual-MiniLM-L12-v2.

4.4.2 출시 직전 실사 — regen_handoff 0/10

동결 직전, 운영점 그대로 dev 양성 40개에 대한 회수율(recall)을 측정했다. 결과: repeat_node 10/10, pingpong_aba 10/10, requery_known 10/10 — 그러나 regen_handoff 0/10, 전체 30/40(75%). 한 패턴이 깔끔하게 전멸한 것은 모델 잡음이 아니라 구조적 누락의 신호였고, 동결을 보류한 채 읽기 전용 진단을 실행했다.

진단 결론: **구조 갭 확정**. regen_handoff는 서로 다른 두 노드(A→B) 사이의 재생성이라 '같은 노드 반복' '핑퐁' 어느 후보 경로에도 걸리지 않는다. 코사인은 0.862로 의미 임계는 훌쩍 넘기에, 후보만 생기면 즉시 잡힌다 — 즉 의미 레이어 문제가 아니라 structural.py에 후보 경로 자체가 없는 것.

4.4.3 원리에 입각한 디스코프 결정

그런데 더 깊은 통찰이 나왔다: regen_handoff는 나머지 셋과 범주가 다르다. repeat_node(같은 노드), requery_known(같은 입력 키), pingpong(A→B→A→B)은 모두 강한 구조 신호가 있지만, regen_handoff는 '인접한 두 에이전트가 각 1회'라는, 사실상 모든 정상 핸드오프와 구조적으로 동일한 모양이다. 후보를 '인접한 서로 다른 노드'로 잡으면 모든 핸드오프가 후보가 되고, 판별이 전적으로 의미 임계에 떠넘겨진다 — IBM 논문이 약하다고 보여준(F1 0.28) 의미-단독 영역, 캐스케이드가 피하려던 바로 그 구간이다. 정당한 다듬기(refinement) 핸드오프는 출력 유사도가 자연히 높아(0.6~0.8) $\phi=0.514$ 를 넘어 오탐될 위험이 컸다.

결정: **regen_handoff를 v1 범위에서 의도적으로 제외(디스코프)**. 단, 디스코프를 합격 구제 수단으로 쓰지 않았다 — 데이터셋에는 그대로 두고, 평가에서 [uncovered]로 투명 표기하며, 집계 F1에는 미탐(FN)으로 불리하게 그대로 반영했다. regen을 짚어진 채로도 0.80을 넘기면 그것

이 가장 강한 결과이기 때문이다.

4.4.4 동결과 단발 평가 — GO

동결 시퀀스(stage2-detector-freeze → eval 재베이스) 후, 전 과정의 규율이 향한 단 한 발 — 평가 세트(seed=42)에 대한 단발 평가를 실행했다. 사격 전에 해석을 미리 못 박았다: 어떤 수치가 나와도 수용, 재튜닝·재생성·파라미터 변경 전부 금지.

- **결과: GO. F1 0.8571(≥ 0.80), FPR 0.0000, 범위 내 TPR 1.0** — TP=30 / FP=0 / TN=40 / FN=10(전부 regen [uncovered]).
- out-of-sample(seed=42)이 dev 결과를 정확히 재현 → φ 가 dev 표본에 과적합되지 않았음까지 확인.
- git 태그: stage1-freeze · stage2-detector-freeze · stage2-eval-freeze · stage2-eval-go. EVAL_RUNS.md 에 run #1 로 기록.

동시에 이 GO가 무엇을 검증했고 무엇이 아닌지도 명확히 새겼다. 검증된 것: 3패턴 캐스케이드가 사전등록된 held-out 합성 트레이스에서 게이트를 통과했다는 것. 아직 아닌 것: 전부 합성 데이터이고, dev/eval이 같은 생성기 출신이라 실세계 일반화는 미검증이며, n=40이라 FPR 신뢰구간이 넓고, 실측 토큰 절감액과 사용자는 0이라는 것.

쉽게 말하면

마지막 고비가 둘 있었습니다. 첫째, 출시 직전 점검에서 네 가지 낭비 유형 중 하나가 전혀 안 잡히는 걸 발견했습니다. 조사해 보니 그 유형은 '정상적인 업무 인계와 겉모습이 똑같아서' 선불리 잡으려 들면 멀쩡한 작업까지 오탐하게 되는 까다로운 놈이었습니다. 그래서 무리하게 끼워넣지 않고 '이번 버전에서는 다루지 않음'을 공개적으로 명시하되, 점수 계산에는 그 실패를 그대로 반영하는 정공법을 택했습니다. 둘째, 그 불리함을 안고도 최종 시험을 한 번에, 수정 없이 통과했습니다(F1 0.857, 오탐 0).

4.5 대화 ⑤ “Custos 프로젝트 종합 현황 분석 및 문서화” (6 월 12 일)

대표가 알아야 할 지식·진행 방향·관련 논문을 종합해 달라는 요청에 따라, 4개 대화를 정독하고 17쪽 분량의 종합 보고서(흑백 차트·다이어그램 7개 포함, 전부 검정 글씨, 각 섹션에 '쉽게 말하면' 해설)를 docx로 제작했다. 핵심 메시지는 본 보고서와 동일하다: 기술 검증은 통과했으나 전부 합성 데이터이며, 다음 한 걸음은 실제 트레이스에서의 오탐 0 유지와 실측 절감액 확보라는 것.

5. 기술 심층 해설

5.1 캐스케이드 아키텍처 — 왜 이 구조인가

Clew의 탐지기는 2단 캐스케이드다. 1단(구조 레이어)이 트레이스의 모양(같은 노드 반복, $A \leftrightarrow B$ 핑퐁, 같은 입력의 재조회)만 보고 '낭비 후보'를 싸고 빠르게 추려낸다. 2단(의미 레이어)이 후보 쌍의 출력 텍스트를 임베딩해 코사인 유사도가 임계값 ϕ 를 넘는지 — 즉 정말로 같은 내용을 또 만든 것인지 — 확인한다.

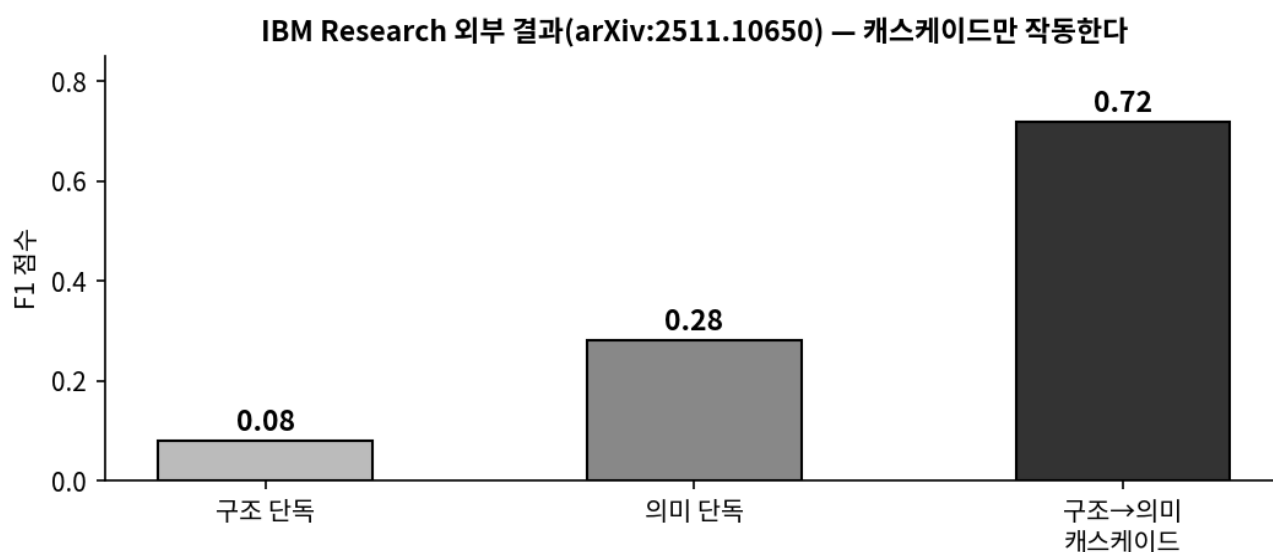


그림 2. IBM Research 외부 결과 — 구조 단독(0.08)도 의미 단독(0.28)도 약하고, 캐스케이드(0.72)만 작동한다. 이 수치는 IBM의 결과이며 Clew의 검증 결과(F1 0.857, 합성 데이터)와 혼동하면 안 된다.

v1이 실패한 이유도 이 그림으로 설명된다: v1은 두 신호를 캐스케이드가 아니라 합산했고, 국소 중복이 아니라 전역 추세를 봤다. 직관은 맞았지만 조립이 틀렸던 것이다.

5.2 네 가지 낭비 패턴

패턴	모양	구조 신호	v1 범위
repeat_node	같은 노드가 같은 출력을 반복 생성	같은 node_id (강함)	포함 — 탐지율 100%
pingpong_ab	$A \leftrightarrow B$ 가 결론 없이 왕복	$A \rightarrow B \rightarrow A \rightarrow B$ 반복 (강)	포함 — 탐지율

		함)	100%
requery_known	이미 조회한 것을 같은 입력으로 재조회	같은 입력 키 (강함)	포함 — 탐지율 100%
regen_handoff	B가 A의 결과를 다시 생성	없음 — 정상 인계와 동형	의도적 제외 [uncovered]

5.3 자기기만 방지 장치 — 이 프로젝트의 영혼

- **사전등록(pre-registration)** — 결과를 보기 전에 GO/KILL 기준과 C1–C4 를 문서로 동결. 결과를 본 뒤 골대를 옮기는 것을 원천 봉쇄.
- **3 중 누출 가드** — AST 정적 분석, 런타임 프로브, 탐지 모듈 비어있음 검사로 탐지 코드가 정답 라벨을 절대 읽을 수 없게 보장.
- **dev/eval 분리**와 **단발 평가** — 파라미터는 dev(seed=7)에서만 결정, 평가 세트(seed=42)는 git 동결 후 평생 단 한 번만 접촉. 결과가 무엇이든 수용.
- **분포 분리 기반 캘리브레이션** — F1 그리드서치(과적합 유혹) 대신 P10/P90 갭과 Cohen's d 로 두 분포가 실제로 갈라지는지 판정.
- **빨간불이면 정지** — 가드가 한 번이라도 켜지면 진행하지 않고 원인을 실측으로 가른 뒤 결정. regen 0/10 때도 동결을 보류하고 진단부터 했다.
- **사람 게이트** — Claude Code 의 플랜 모드와 수동 승인 모드로, git 동결·파라미터 동결·단발 평가 같은 비가역 단계는 반드시 사람이 최종 승인.

5.4 최종 결과 상세

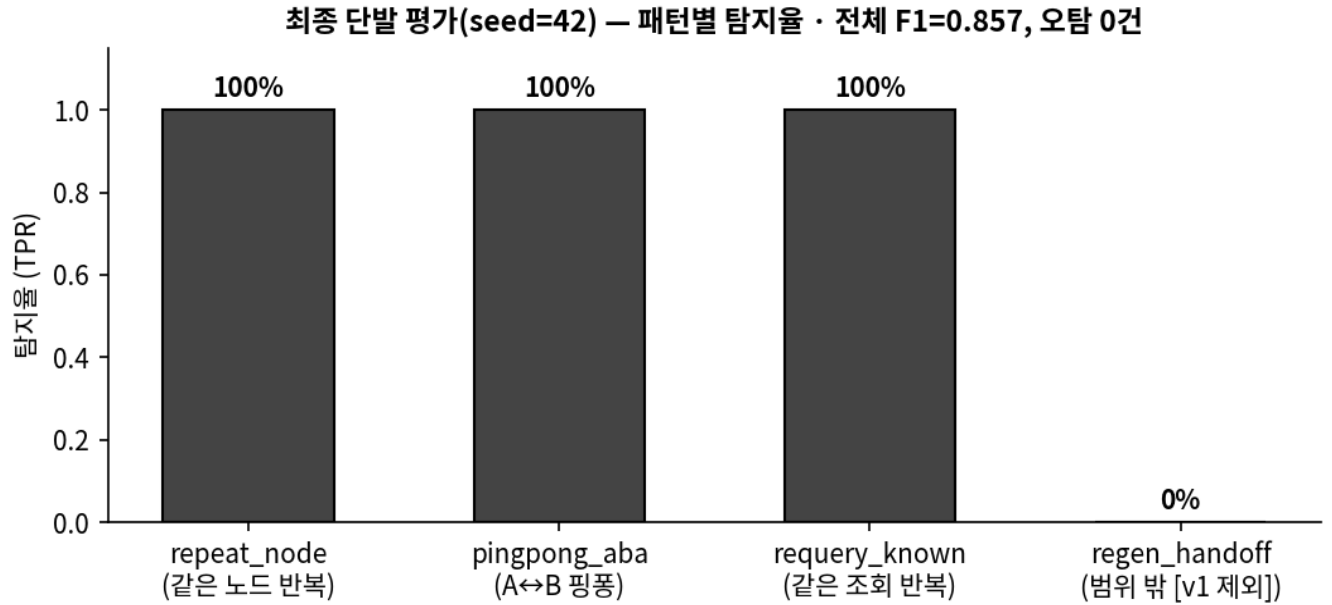


그림 3. 단발 평가(seed=42) 패턴별 탐지율 — 범위 내 3패턴 100%, regen은 의도적 제외.

지표	값
F1 (집계, regen 미탐을 불리하게 포함)	0.8571 (합격선 0.80 이상)
거짓 양성(FP)	0건 / FPR 0.0000
혼동 행렬	TP=30, FP=0, TN=40, FN=10(전부 regen)
동결 파라미터	$\phi=0.514345$, N=2, paraphrase-multilingual-MiniLM-L12-v2
dev 분리 지표	gap +0.2208, Cohen's d 4.3803
테스트	146 통과 (1 의도된 xfail)
git 태그	stage1-freeze, stage2-detector-freeze, stage2-eval-freeze, stage2-eval-go

쉽게 말하면

이 장의 핵심은 두 가지입니다. 첫째, '모양으로 먼저 거르고 내용으로 확정한다'는 2단 구조가 이 제품의 기술적 심장이라는 것. 둘째, 점수 자체보다 '그 점수를 조작이 불가능한 절차

로 얻었다'는 사실이 더 값지다는 것입니다. 투자자나 고객에게도 후자가 신뢰의 근거가 됩니다.

6. 관련 논문 및 문서

6.1 핵심 외부 논문

① IBM Research — arXiv:2511.10650

멀티에이전트 트레이스에서 중복 낭비를 탐지하는 방법을 비교한 연구. 구조 신호 단독은 F1 0.08, 의미 유사도 단독은 F1 0.28에 그치지만, 구조가 후보를 추리고 의미가 확정하는 캐스케이드는 F1 0.72를 달성함을 보였다. Clew 아키텍처의 직접적 근거다.

주의: 이 수치는 IBM의 데이터 구현에 대한 외부 결과다. Clew의 F1 0.857(자체 합성 데이터)과 절대 섞어 말하지 않는다는 원칙이 모든 공개 커뮤니케이션에 적용된다.

② MAST — arXiv:2503.13657 (Cemri et al., UC Berkeley)

멀티에이전트 시스템 실패 분류 체계(Multi-Agent System Failure Taxonomy)와 그 데이터셋. v1 검증 실험의 데이터 기반이었다(Hugging Face mcmri/MAST-Data, 7개 프레임워크, 1,242건 레코드 — 공개 글에서는 '1,600+' 표기 이슈가 있어 정확 수치 확인 후 통일하기로 함). 실패 유형의 공인 분류를 제공하므로 향후 제품의 패턴 확장 로드맵에도 참조 축이 된다. GitHub: [multi-agent-systems-failure-taxonomy/MAST](https://github.com/mcmri/multi-agent-systems-failure-taxonomy/MAST).

③ 표준·생태계 문서

- OpenInference / OpenTelemetry(OTel) — Clew의 정규 스팸 트리 모델이 따르는 트레이싱 표준. 이 표준을 따르므로 LangGraph 등 주요 프레임워크의 트레이스를 어댑터로 수용할 수 있다.
- v1 EmergenceTrace 연구보고서 — 17개 참고문헌을 포함한 자체 문서. 문제 정의, Emergence Score 방법론, 기술 아키텍처 9개 절, 검증 계획, 리스크 분석 수록.

6.2 내부 핵심 문서 (프로젝트 폴더)

문서	역할
SPEC.md	로드맵(§7)과 현재 스테이지 상세(§8). 모든 빌드의 단일

	진실 원천
CLAUDE.md	Claude Code가 매 세션 자동으로 읽는 상시 컨텍스트. 현재 스테이지 포인터
validation/CRITERIA_FROZEN.md	사전등록·동결된 성공/폐기 기준 (GO/KILL, C1-C4)
CALIBRATION_LOG.md	캘리브레이션 이력 — 실패·진단·수정의 전 기록
EVAL_RUNS.md	평가 실행 대장. run #1 GO 기록
Clew_프로젝트_전체기록_마케팅핸드오프.md	SNS/마케팅용 핸드오프 — 검증 경계와 비협상 커뮤니케이션 규칙 명시

쉽게 말하면

외부 논문 두 편이 기둥입니다. IBM 논문은 '어떻게 만들어야 하는가'(캐스케이드)를, 버클리 MAST는 '무엇이 실패인가'(실패 분류와 실제 데이터)를 줍니다. 그리고 내부 문서 6개는 '우리가 스스로를 속이지 않았다'는 증거 서류철입니다.

7. 시장과 경쟁

7.1 시장 규모와 성장

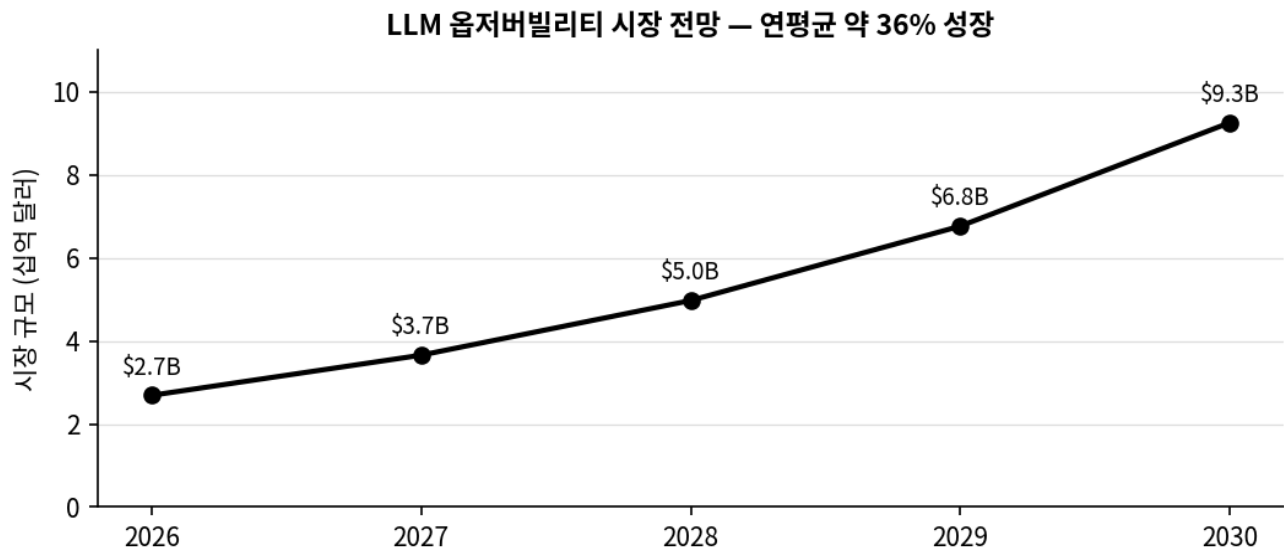


그림 4. LLM 옵저버빌리티 시장 전망 (\$2.69B → \$9.26B, 2026→2030)

- 기업 AI 의 72%가 멀티에이전트 구성(2024 년 23%에서 1 년여 만에 3 배 이상).
- 기존 도구 만족도 33% 미만 — 시장은 크지만 풀리지 않은 고통이 많다.
- 토큰 비용 폭증이 기업들의 1 순위 운영 고통 — Clew 의 빼기가 정확히 여기를 겨냥.

7.2 경쟁 지형

진영	대표 주자	Clew와의 관계
APM 계열	Datadog, New Relic	범용 관측. 에이전트 흐름 차트까지 진출했으나 '낭비=비용' 특화는 아님
AI 네이티브 추적	LangSmith, Langfuse(ClickHouse 인수, \$400M), Laminar, Arize/Phoenix	트레이스 수집·디버깅이 중심. Clew는 그 위의 분석 레이어로 보완 가능
게이트웨이	Helicone, Portkey	호출 단의 비용 제어. 트레이스 구조 분석은 아님

포지셔닝 원칙: 경쟁사를 깎아내리지 않고 '다른 레이어, 다른 규율'로 위치 잡는다. 수집은 그들이 잘하고, Clew는 수집된 트레이스에서 돈이 새는 구간을 짚는 분석에 집중한다. 또한 '프레임 워크 무관'은 차별점이 아니라 입장권(table stakes)이라는 1차 대화의 교훈을 유지한다.

쉽게 말하면

시장은 빠르게 크고 있고(4년에 3.4배), 경쟁자도 많습니다. 하지만 큰 회사들은 '모든 것을 보여주는 대시보드' 싸움을 하고 있고, Clew는 '이 구간에서 매달 얼마가 새고 있습니다'라는 한 문장 싸움을 하려는 것입니다. 작은 팀이 이길 수 있는 싸움은 후자입니다.

8. 이 사업에 대한 솔직한 의견

8.1 강점

- **검증 규율 자체가 자산** — 사전등록·단발 평가·누출 가드는 흔치 않은 실행 품질의 증거다. 기술 실사(due diligence)에서 강력한 신뢰 신호가 되고, build-in-public 콘텐츠로서도 차별적이다.
- **검증된 고통에 정확히 쫓힌 썰기** — ‘토큰 비용’은 추측이 아니라 시장조사로 확인된 1 순위 고통이고, 낭비 탐지는 ROI 를 달러로 환산해 보여줄 수 있는 드문 카테고리다.
- **외부 연구가 받쳐주는 아키텍처** — 캐스케이드 접근은 IBM 연구로 독립 검증된 방향이라 ‘혼자만의 믿음’ 리스크가 낮다.
- **극도로 낮은 소진율** — 1 인 + Claude Code 개발 체제로 열흘 만에 검증 사이클 완주. 자본 효율이 매우 높다.

8.2 약점과 리스크

- **합성 데이터의 한계 (최대 리스크)** — FP 0·TPR 100%의 완벽함은 깔끔하게 분리된 합성 분포의 성질이다. 실제 트레이스는 토폴로지·출력·잡음이 훨씬 지저분하며, 특히 정당한 다듬기 핸드오프가 오염을 유발할 가능성이 있다. 실전에서 깨질 수 있다.
- **수요 미검증** — 사용자 0 명. ‘낭비를 보여주면 돈을 낼 것’이라는 가정은 아직 한 명의 고객으로도 확인되지 않았다.
- **거대 경쟁자의 번들링 위협** — Datadog·클라우드 3 사가 같은 기능을 기존 제품에 끼워 넣으면 단독 제품의 입지가 좁아진다. 속도와 특화, 커뮤니티가 방어선이다.
- **1 인 팀 리스크** — 개발·영업·마케팅을 혼자 감당해야 하며, 일부 투자사(특히 YC)는 단독 창업자를 상대적으로 불리하게 본다.
- **커버리지 공백** — regen_handoff 는 원리적으로 까다로운 패턴이라 v2 과제로 남아 있다. 경쟁사가 이 지점을 파고들 수 있다.

8.3 종합 판단

이 사업은 현재 '기술 가설은 1차 검증, 사업 가설은 미검증' 상태다. 좋은 소식은 가장 싸게 확인할 수 있는 것부터 순서대로 확인해 왔다는 점이고, 나쁜 소식은 가장 비싼 확인(실제 고객·실제 데이터·실제 절감액)이 아직 전부 남아 있다는 점이다. 다만 그 다음 확인도 '가장 작은 진짜 산출물' 원칙으로 설계되어 있어, 실패하더라도 빠르고 정직하게 알 수 있는 구조다. 솔직한 평가로는 — 방법론 면에서 상위 1% 수준의 초기 스타트업이며, 시장 타이밍도 좋다. 관건은 향후 4~8주의 실전 검증이 합성의 완벽함을 얼마나 보존하느냐다.

쉽게 말하면

비유하면, 새로 개발한 그물(탐지기)이 수조 실험에서는 물고기를 다 잡고 멀쩡한 물체는 하나도 안 건드렸습니다. 하지만 아직 바다에 나가본 적이 없습니다. 바다(실제 기업 데이터)에서는 해초(정당한 작업)가 그물에 걸릴 수 있습니다. 다음 항해가 이 사업의 진짜 시험입니다.

9. 다음 방향 — 앞으로의 로드맵

9.1 즉시 (1~2 주): 실제 트레이스 첫 접촉

- 실제 멀티에이전트 트레이스 확보 — 자신의 Claude Code 세션 로그, 공개 LangGraph/에이전트 트레이스 등에서 시작.
- 거기서 확인할 세 가지: (a) 후보가 합리적으로 잡히는가, (b) 거짓 양성이 합성처럼 0에 가깝게 유지되는가 — 여기서 깨질 가능성이 가장 높다(실전의 다듬기 핸드오프가 ϕ 를 건드릴 수 있음), (c) 잡은 낭비의 실측 토큰 절감액이 얼마인가.
- (c)가 비용 스토리의 핵심이다. “토큰을 태우는 낭비를 잡아 줄인다”는 사업 주장은 합성 F1이 아니라 실제 절감 숫자로만 뒷받침된다.

9.2 단기 (2~6 주): 첫 사용자와 실측 숫자

- 준비된 GTM 초안 실행 — DEV.to의 정직한 실험기(v1 실패담)부터 발행. ‘불확실성 노출이 신뢰 메커니즘’이라는 톤 유지.
- 디자인 파트너 3~5팀 확보 — 멀티에이전트를 프로덕션에서 돌리는 팀의 트레이스를 받아 무료 진단 리포트 제공. 콜드 DM 템플릿과 커뮤니티 댓글 전략 활용.
- 첫 실측 케이스 스터디 1건 — “X팀 트레이스에서 월 \$Y의 낭비를 발견”한 문장을 만드는 것이 이 구간의 단 하나의 목표.

9.3 중기 (6주~3개월): 제품화와 v2

- CLI/라이브러리 형태의 최소 제품 — 트레이스 파일 입력 → 낭비 리포트 출력. Langfuse/LangSmith 내보내기 포맷 어댑터 추가로 설치 마찰 최소화.
- regen_handoff v2 실험 — 정당한 refinement 트윈(높은 코사인의 비낭비)을 hard negative로 추가한 정직한 실험 설계. FP 표면이 깨지면 그것대로 정직한 음성으로 기록.
- 실전 데이터 기반 재캘리브레이션 — 단, 동일한 사전등록·단발 평가 규율 유지.

9.4 전략 원칙 (변하지 않는 것)

- 모든 공개 숫자에 맥락(합성·held-out-3 패턴) 부착. IBM 결과를 우리 성과로 말하지 않음.
- 사용자·실측 절감이 없음을 숨기지 않음 — build-in-public 에서는 '아직 없다' 자체가 콘텐츠.
- 가장 위험한 가정을 시험하는 가장 작은 진짜 산출물만 만든다.

쉽게 말하면

다음 4~8주의 목표는 단 한 문장을 얻는 것입니다: "실제 회사 트레이스에서 월 얼마의 낭비를 찾아냈다." 이 문장이 생기는 순간 영업 자료, 투자 피치, 콘텐츠가 전부 한 단계 올라갑니다. 반대로 이 문장이 안 생기면 — 그것도 빨리 아는 것이 이 회사의 방식입니다.

10. 투자 유치 가능성 분석 — YC 를 중심으로

10.1 시장의 투자 분위기

에이전트 인프라는 현재 초기 투자 시장에서 가장 뜨거운 카테고리 중 하나다. YC 최근 배치 (W26) 분석에 따르면 배치의 약 60%가 AI 기업이고, 약 41.5%가 에이전트 아래의 배관(인증·테스트·보안·모니터링·컨텍스트 관리·과금)을 만들고 있으며, 관측(observability) 레이어 스타트업이 다수 포함되어 있다. YC Fall 2025 배치 분석에서도 '에이전트 파일럿이 드러낸 인프라 공백'이 핵심 테마였고, 오피저버빌리티 도구가 명시적 하위 카테고리로 꼽혔다. 즉 Clew는 정확히 지금 돈이 몰리는 카테고리에 서 있다.

10.2 YC 관점에서 본 Clew

유리한 점

- **카테고리 적합** — 에이전트 인프라/오피저버빌리티는 YC 가 최근 배치에서 가장 많이 뽑는 영역이다.
- **YC 가 사랑하는 서사** — '가설을 사전등록 실험으로 죽이고, 정직하게 피벗해서, 열흘 만에 검증 게이트를 통과시켰다'는 이야기는 YC 가 강조하는 학습 속도(rate of learning)와 지적 정직성의 교과서적 사례다. 인터뷰에서 매우 강하게 작동할 수 있다.
- **측정 가능한 가치 제안** — '절감된 토큰 = 절감된 달러'라는 직접적 ROI 는 YC 파트너들이 선호하는 명료한 가치 명제다.
- **자본 효율** — 1 인 + AI 도구로 이 속도를 내는 것 자체가 새 시대의 팀 생산성 증거다.

불리한 점 (지원 전에 보강할 것)

- **트랙션 부재** — 사용자 0, 매출 0, 실측 절감 0. YC 는 아이디어 단계도 뽑지만, 이 카테고리는 경쟁 지원자가 많아 작은 트랙션이라도 큰 차이를 만든다. 디자인 파트너 3~5 팀 + 실측 케이스 1 건이 지원서의 체급을 바꾼다.
- **단독 창업자** — YC 는 공동창업 팀을 선호하는 경향이 뚜렷하다. 단독으로도 합격 사례는 있으나, '혼자서도 이 속도'를 입증하는 지표(주간 출시 로그, 사용자 성장)가 더 필요하다.

- **뽐비는 카테고리** — 같은 배치 안에서도 옹저버빌리티 스타트업이 여럿이다. '뽐뽐 관측이 아니라 뽐뽐-뽐뽐 특화'라는 한 문장 차별화를 지원서 첫 줄에 박아야 한다.

10.3 YC 외 옵션

트랙	대상	적합성 메모
미국 액셀러레이터	Y Combinator, Neo, South Park Commons	카테고리 적합 최상. 트랙션 1건 확보 후 지원 권장
AI 특화 초기 VC	a16z(Infra), Felicis, Boldstart, Uncork 등 데브툴/인프라 시드	실측 절감 케이스 + 디자인 파트너 로고 가 생기면 프리시드 대화 가능
전략적 투자/인수자 관점	ClickHouse(Langfuse), Datadog, Arize	장기적으로 인수 출구의 자연스러운 후보군 — 지금은 파트너십·통합 대상으로 접근
한국 트랙	TIPS(팁스), 프라이머, 매쉬업엔젤스, 스파크랩스, 퓨처플레이	서울 기반 1인 창업자에게 현실적 첫 자금원. TIPS는 R&D 자금 + 지분 희석 최소화. 글로벌 B2B SaaS 지향성을 명확히 하면 가산점
비희석 자금	정부 과제(창업지원), 클라우드 크레딧(AWS/GCP/Azure/Anthropic 스타트업 프로그램)	런웨이 연장용. 소진율이 낮아 효과가 큼

10.4 투자 준비 체크리스트

- 실측 케이스 1 건: "실제 트레이스에서 월 \$X 뽐뽐 발견" — 모든 피치의 첫 슬라이드.
- 디자인 파트너 3~5 팀의 이름(또는 익명화된 업종·규모).
- 주간 빌드 로그 공개(build-in-public) — 단독 창업자 리스크에 대한 반박 증거.
- 정직성 서사를 1 분 스토리로 압축: AUC 0.455 → 피벗 → 단발 평가 GO.
- 숫자 위생 유지: 합성/실측, 내부/외부(IBM) 결과를 절대 섞지 않기 — 실사에서 신뢰의 근간.

쉽게 말하면

결론부터 말하면: 지금 당장 지원해도 '이야기'로는 경쟁력이 있지만, 4~8주 안에 실제 고객 데이터에서 절감 사례 1건만 만들면 합격 확률이 크게 달라집니다. YC가 최근 뽑는 회사의 절반 가까이가 바로 이 '에이전트 배관' 영역이라 문은 열려 있습니다. 다만 단독 창업자 + 무(無)트랙션 조합은 가장 흔한 탈락 사유이므로, 트랙션 한 조각을 먼저 만드는 순서를 권합니다. 한국에서는 TIPS·프라이머 같은 트랙이 희석 없이 런웨이를 늘려주는 현실적 선택지입니다.

11. 결론

Custos/Clew는 열흘 만에 '비판 → 검증 → 정직한 실패 → 피벗 → 빌드 → 사전등록 게이트 통과'의 전체 사이클을 완주했다. 현재 위치를 한 문장으로 말하면: **"심어둔 3가지 낭비 패턴을, 튜닝에 쓰지 않은 합성 트레이스에서 거짓 양성 0으로 잡는 캐스케이드를, 사전등록 기준으로 통과시켰다."**

이 문장에서 '합성'이라는 단어를 '실제'로 바꾸고, 그 옆에 달리 표시가 붙은 절감액 하나를 세우는 것 — 그것이 다음 4~8주의 전부다. 그 숫자가 생기면 영업·콘텐츠·투자(YC 포함)가 동시에 열리고, 생기지 않으면 이 회사의 방식대로 빠르고 정직하게 알게 될 것이다. 어느 쪽이든, 이 프로젝트가 이미 증명한 가장 귀한 것은 '자기를 속이지 않고 빠르게 배우는 시스템'이며, 그것은 어떤 피벗에도 이월되는 자산이다.

비협상 커뮤니케이션 규칙(상기): (1) '프로덕션 토큰 절약'이 아니라 '합성 검증에서 FP 0'으로 말한다. (2) IBM F1 0.72는 외부 결과로만 인용한다. (3) '4패턴'이 아니라 '3패턴 + regen 명시적 제외'로 말한다. (4) 모든 숫자에 맥락을 붙인다. (5) 사용자·실측 절감이 없음을 숨기지 않는다. (6) 경쟁사를 깎지 않는다.